

Occupancy-Corrected Sparse Representation for Cardinality Estimation

Luca Cappelletti*
University of Fribourg

June 30, 2026

Keywords: cardinality estimation, HyperLogLog, streaming algorithms, sparse representation, hash list, occupancy correction, space-efficient counting

Abstract

HyperLogLog and its successors maintain an array of $m = 2^P$ registers, each storing the maximum observed rank, and estimate cardinality from their harmonic-mean estimator. HLL achieves relative error $1/\sqrt{m}$ only at high cardinality. At low cardinality ($n \lesssim 2^{P+1}$), the estimate is dominated by sampling noise and existing systems rely on linear counting or empirically fitted correction tables. Sparse representations (storing explicit hash values instead of a fixed register array) extend the near-exact range. We build on this direction with a three-stage adaptive counter that progresses from an exact value list to a hash list of distinct composites, pushing the sparse representation deep into the collision regime where the raw distinct count is biased low. We correct this bias by inverting an analytically derived occupancy function via safeguarded Newton iteration. At precision $P = 10$ and register width $B = 6$ bits, the corrected hash-list estimate achieves mean absolute relative error of 0.86% over its regime (compared to 2.20% for linear counting, 1.98% for register MLE, and 3.83% for SetSketch with the same bucket count), with zero error for the value-list portion of the range. The counter converts to HyperLogLog registers when the hash list saturates. This three-stage design (value list, hash list, dense representation) provides a general template for register-based estimators that lack low cardinality accuracy. The default counter uses fixed-size stack arrays and is available as an open source crate.¹

1 Introduction

Cardinality estimation under space constraints is a foundational problem in data streaming, database systems, and distributed computing. The HyperLogLog (HLL) algorithm of Flajolet et al. [1] hashes each element of a stream and maintains an array of $m = 2^P$ small integer registers, each storing the maximum observed rank (number of leading zeros plus one) among elements hashing to that bucket. The cardinality estimate derives from the harmonic-mean estimator and achieves relative standard error $\sigma \approx 1.04/\sqrt{m}$ in the asymptotic regime ($n \gg m \log m$).

This asymptotic guarantee masks significant error at low cardinality. Below $n \approx 2^{P+1}$, many registers remain empty and the harmonic-mean estimator is dominated by the m zero entries, yielding large relative error. The standard remedy is linear counting ($\hat{n} = -m \ln(Z/m)$ where Z

*luca.cappelletti@unifr.ch, <https://orcid.org/0000-0002-1269-2038>

¹<https://github.com/LucaCappelletti94/hyperloglog-rs>

is the number of zero registers), which is table-free and accurate only in a narrow band near the transition.

This low-cardinality regime is the operating point of many real-world workloads. In graph analytics, many network classes exhibit heavy-tailed degree distributions (power-law or scale-free graphs) in which a large fraction of nodes have small degree (often single digits to low hundreds). The HyperBall algorithm of Boldi and Vigna [9] approximates geometric centralities (closeness, harmonic centrality) on very large graphs by maintaining per-node HLL counters during ball expansion. On heavy-tailed graphs, the per-node HLL counter operates in the low-load regime for many nodes, where the asymptotic guarantee does not apply and empirical bias dominates. Our adaptive three-stage representation addresses this limitation: many nodes remain in the exact value list or the near-exact hash list, and only nodes upgrade to the probabilistic register array. This avoids paying the cost of probabilistic estimation for nodes that do not need it.

HLL++ uses a sparse representation (sorted compressed index-rank pairs) for very low cardinality, then linear counting in the next band, then empirically fitted bias-correction tables for the intermediate regime. Our counter operates in the linear-counting band: it maintains explicit representations of the inserted data until space pressure requires compression, progressing through three stages:

1. **Value list.** For the smallest sets, the exact sorted list of inserted values is stored with gap compression: the first value is encoded absolutely and each subsequent value as the gap from its predecessor minus one, using Elias-gamma coding. Subtracting one from each gap is always valid (distinct sorted values have gaps of at least one), and it makes the common case of contiguous values cost a single bit each (gap one encodes as zero). The sorted order serves a dual purpose: it enables fast set operations (intersection, union) via merge-style traversal, and it minimizes the gap codes between consecutive values. The effective capacity of the value list depends on the gap distribution of the input data: in the best case (contiguous values), each element after the first requires a single bit, yielding an upper bound equal to the buffer size in bits. When the input values follow a global ordering (for example, graph node IDs produced by a Layered Label Propagation (LLP) reordering [8] that clusters nodes at multiple resolutions and produces an ordering where topologically close nodes receive nearby IDs), the gaps are small and capacity approaches this bound. Cardinality is exact.
2. **Hash list.** When the value list fills, each value is replaced by a composite hash: a w -bit fingerprint, where w is the current composite width, concatenating the P -bit bucket index, the register rank, and residual hash bits. The sorted list of distinct composites is stored with Rice-coded gaps between consecutive entries. Cardinality is estimated by inverting an analytically derived occupancy function: the expected number of distinct composites as a function of cardinality (a birthday-paradox problem with non-uniform cell probabilities), inverted via safeguarded Newton iteration to recover the cardinality estimate.
3. **Dense representation.** When the hash list saturates, the counter converts to a representation optimized for large cardinality. In the current implementation, this is the standard HLL register array.

The novelty of this work lies in the Rice-coded gap packing of the hash list and the occupancy-inverse bias correction. The packing pushes cardinality deep enough into the collision regime to make the correction necessary and impactful. A detailed comparison with HLL++’s sparse representation follows in section 5.

The remainder of this paper formalizes the method (section 2), derives the occupancy model and its inversion (section 3), presents empirical results (section 4), relates the approach to HLL++ (section 5), and discusses applicability to SetSketch and HyperLogLogLog (section 6).

2 Method

2.1 Composite Hash Encoding

Each element x is hashed by a uniform 64-bit hash function $h(x)$. The top P bits select the register bucket index $I(x) = \lfloor h(x)/2^{64-P} \rfloor$. The register rank $R(x) = \rho(h(x) \bmod 2^{64-P}) + 1$ is the number of leading zeros of the tail plus one, capped at $2^B - 1$. The composite hash at width $t = w - P$ tail bits takes three forms depending on t and the register value:

The composite hash packs three pieces of information into w bits: the bucket index, the register rank, and extra hash bits for differentiation. Because the register rank follows a geometric distribution (small values are common), we store it explicitly only when it is large, and use the extra hash bits as a proxy when it is small. This saves space for the common case. Formally:

Definition 2.1 (Composite hash forms). *Let w be the current composite width and $t = w - P$ the tail width. The composite $C(x)$ is a w -bit string:*

- *If $t = B$ (smallest width), $C(x)$ concatenates the P -bit index and the B -bit register value. No residual bits remain.*
- *If $t > B$ and the register value $r \leq B + 1$, a flag bit 0 is followed by $t - 1$ leading hash bits (the most significant bits of the original hash tail). The register information is implicit: r is small enough that the leading bits carry sufficient resolution.*
- *If $t > B$ and $r > B + 1$, a flag bit 1 is followed by the B -bit register value and $t - 1 - B$ residual hash bits.*

The width w starts at a large value (e.g. 24 bits for $P = 10$) and decreases as the list fills. At the maximum width, distinct elements almost never collide (the composite space is $2^w \gg n$). As w shrinks, the collision rate rises and the number of distinct composites D begins to undercount n . The correction for this undercount is the occupancy inversion described in section 3.

2.2 Rice-Coded Gap Compression

The composite hashes are maintained in sorted order. Instead of storing each w -bit composite explicitly, we store the gap from each composite to its predecessor, encoded with a Rice code [7]. A Rice code with parameter k represents an integer x as the quotient $\lfloor x/2^k \rfloor$ in unary followed by the k -bit remainder. The expected code length is minimized when $2^k \approx \mu \ln 2$ where μ is the mean gap.

Lemma 2.2 (Optimal Rice parameter). *For D distinct composites uniformly distributed over a 2^w space, the mean gap is $\mu = 2^w/D$. The optimal Rice parameter for a geometric gap distribution is*

$$k \approx \log_2(\mu \ln 2) = w - \log_2 D - 0.53.$$

Proof. The gaps between sorted uniform order statistics are geometrically distributed with mean $\mu = 2^w/D$. Golomb coding is optimal for a geometric source [6], and Rice coding is Golomb restricted to a power-of-two parameter $M = 2^k$. The optimal Golomb parameter for geometric mean μ is $M \approx \mu \ln 2$, hence $k = \lceil \log_2(\mu \ln 2) + 0.5 \rceil$. Substituting $\mu = 2^w/D$ gives $k \approx w - \log_2 D - 0.53$. $\square$

To obtain a closed form, we substitute the steady-state occupancy observed empirically: when the width downgrade triggers, the distinct count clusters near $D \approx 2^{P-1.53}$ under uniform hashing. This yields the practical approximation $k \approx w - P + 1$, which depends only on the current width and precision. The implementation uses $k = w - P + 1$ for $w > P + B$ and $k = 0$ at the narrowest width, matching the empirically optimal parameter within one step for all tested configurations.

The Rice parameter is derived analytically from the encoding structure, no per-configuration calibration or lookup table is required. The compression ratio is substantial: at $P = 10$ and $w = 24$, each composite requires approximately 14 bytes in the value list (Elias-gamma coded gaps between full 64-bit values spread across the 2^{64} domain) but only 3 to 4 bytes in the sparse representation (Rice-coded gaps between composites in a space of 2^{24}).

2.3 Representation Transitions

This compression enables the counter to defer register conversion, but representation changes must be managed. The counter switches between representations at fixed memory thresholds. The value list holds at most $N_{vl} \approx 200$ elements for $P = 10$ with uniformly distributed input hashes, but actual capacity varies with the gap distribution of the encoded data (higher for clustered or ordered inputs). When a new element would overflow the value list, all stored values are hashed and the counter switches to the hash list at the maximum width $w_{\max}$. The hash list holds elements until the composite width reaches $w_{\min} = P + B$. At that point, the distinct composites are materialized into the dense representation. In the current implementation, the composites are folded into the $m = 2^P$ HLL registers and the counter enters the standard HLL regime.

The default counter uses fixed-size stack arrays sized at construction from (P, B) and requires no heap allocation.

The counter supports allocation-free set operations. Merging two value lists uses a linear two-pointer traversal of the sorted exact values, producing an exact union. Merging two hash lists merges distinct composites at the common (coarser) width, re-encodes the result with Rice-coded gaps, and re-applies the occupancy inversion for the union estimate. At the register level, the merge reduces to the standard element-wise maximum of the two HLL arrays.

3 Occupancy Model and Bias Correction

3.1 Expected Distinct Composite Count

The core of the bias correction is the expected number of distinct composite values D as a function of cardinality n and width w . This is a birthday-paradox occupancy problem: each of n elements lands in one of 2^w composite “boxes,” and D is the number of non-empty boxes. Because the composite encoding is not uniform (index bits are uniform, but register bits follow a geometric distribution and residual bits are uniform), the cell probabilities are not equal. We group cells by probability and sum over groups.

Theorem 3.1 (Occupancy function). *Let $m = 2^P$ and $R = 2^B - 1$ be the maximum register value. We assume that h is a uniform hash function: each of the 2^{64} possible hash values is equally likely for every input element. The expected distinct composite count at cardinality n and width w is*

$$\mathbb{E}[D \mid n, w] = \sum_j g_j (1 - (1 - p_j)^n),$$

where the sum ranges over cell groups (g_j, p_j) defined by the composite encoding:

- At the smallest width ($t = B$), each non-saturating register value $r \in \{1, \dots, R-1\}$ (where $R-1 = 2^B - 2$) corresponds to $g_r = m$ cells of probability $p_r = 2^{-(P+r)}$. The saturating register $R = 2^B - 1$ corresponds to $g_R = m$ cells of probability $p_R = 2^{-(P+2^B-2)}$. Hence

$$\mathbb{E}[D \mid n] = m \left(\sum_{r=1}^{R-1} (1 - (1 - 2^{-(P+r)})^n) + (1 - (1 - 2^{-(P+2^B-2)})^n) \right). \quad (1)$$

- At wider widths ($t > B$), the flag bit partitions the tail space. Flag 0 (register $r \leq B+1$, leading bits stored) has $2^{t-2-\ell}$ cells of probability $2^{-(w-1)}$ for each $\ell \in \{0, \dots, \min(B, t-2)\}$. Flag 1 (register $r > B+1$, register plus residual stored) has 2^{t-1-B} cells of probability $2^{-(P+r+t-1-B)}$ for each $r \in \{B+2, \dots, R-1\}$, with the saturating register at probability $2^{-(P+R-1+t-1-B)}$. Hence

$$\begin{aligned} \mathbb{E}[D \mid n, w] = m \left(\sum_{\ell=0}^{\min(B, t-2)} 2^{t-2-\ell} (1 - (1 - 2^{-(w-1)})^n) \right. \\ \left. + \sum_{r=B+2}^{R-1} 2^{t-1-B} (1 - (1 - 2^{-(P+r+t-1-B)})^n) \right. \\ \left. + 2^{t-1-B} (1 - (1 - 2^{-(P+R-1+t-1-B)})^n) \right). \quad (2) \end{aligned}$$

Proof. Each element independently selects a composite value. The probability that a given cell with probability p_j is occupied by at least one of n elements is $1 - (1 - p_j)^n$. Summing over all cells, grouped by equal probability, gives the result. The cell groups follow from the composite encoding structure. The index bits are uniform, contributing the factor $m = 2^P$ to every group. For the tail bits, the register distribution is geometric: the probability of rank r is 2^{-r} for $r < 2^B - 1$ and $2^{-(2^B-2)}$ for the saturating register. At the smallest width ($t = B$), the tail consists of the B -bit register value, so each register value r corresponds to exactly m cells (one per index), each with probability $2^{-(P+r)}$ for $r < 2^B - 1$ and $2^{-(P+2^B-2)}$ for the saturating register. At wider widths ($t > B$), the flag bit partitions the tail. Flag 0 stores $t-1$ leading hash bits for registers $r \leq B+1$. For a given number ℓ of leading zeros in the stored hash bits ($\ell \in \{0, \dots, \min(B, t-2)\}$), there are $2^{t-2-\ell}$ distinct bit patterns, each defining a cell with probability $2^{-(w-1)}$. Flag 1 stores the B -bit register value and $t-1-B$ residual hash bits for registers $r > B+1$. Each such register defines a group of 2^{t-1-B} cells with probability $2^{-(P+r+t-1-B)}$. The non-uniformity in cell probabilities arises from the geometric register distribution and the flag-bit partition at wide widths. $\square$

3.2 Occupancy Inversion

The estimator $\hat{n}$ is the inverse of the occupancy function at the observed distinct count D :

$$\hat{n} = g^{-1}(D) \quad \text{where} \quad g(n) = \mathbb{E}[D \mid n, w]. \quad (3)$$

We solve $g(n) = D$ by safeguarded Newton iteration. The derivative required for the Newton step is

$$g'(n) = \sum_j g_j (-\ln(1 - p_j)) (1 - p_j)^n, \quad (4)$$

which is always positive (the occupancy function is strictly increasing), ensuring convergence. The safeguard maintains a bracket $[\text{low}, \text{high}]$ initialized at $[D, D]$ and expanded until $g(\text{high}) \geq D$.

Each Newton step $n_{\text{next}} = n - (g(n) - D)/g'(n)$ is accepted only if it falls within the bracket, otherwise, a bisection step is taken. Convergence is declared when the relative step falls below 10^{-12} .

Because D is a random variable and the inversion g^{-1} is nonlinear, Jensen's inequality implies that the estimator carries a small bias: $\mathbb{E}[g^{-1}(D)] \neq g^{-1}(\mathbb{E}[D]) = n$. The empirical MRE values in Table 1 (all near zero) confirm that this bias is negligible in practice for the configurations tested.

The predicted variance of the estimator uses a delta-method approximation that treats the multinomial cell indicators as if they were independent. For the non-uniform cell probabilities of the composite encoding, this is an approximation: the exact variance requires cross terms between cells of different probability. The approximation is accurate to within a few percent for the configurations tested, as confirmed by comparing predicted and empirical standard errors.

Algorithm 1 Occupancy-inversion estimator

Require: Observed distinct count D , current width w

Ensure: Cardinality estimate $\hat{n}$

```

1: Compute cell groups  $(g_j, p_j)$  from encoding at width  $w$ 
2: function OCCUPANCY( $n$ )
3:    $E \leftarrow \sum_j g_j (1 - (1 - p_j)^n)$ 
4:   return  $E$ 
5: end function
6: function OCCUPANCYDERIVATIVE( $n$ )
7:    $E' \leftarrow \sum_j g_j (-\ln(1 - p_j))(1 - p_j)^n$ 
8:   return  $E'$ 
9: end function
10: low  $\leftarrow D$ , high  $\leftarrow D$ 
11: while Occupancy(high) <  $D$  do
12:   high  $\leftarrow 2 \cdot$  high
13: end while
14:  $n \leftarrow (\text{low} + \text{high})/2$ 
15: repeat
16:   if Occupancy( $n$ ) >  $D$  then
17:     high  $\leftarrow n$ 
18:   else
19:     low  $\leftarrow n$ 
20:   end if
21:    $c \leftarrow n - (\text{Occupancy}(n) - D)/\text{OccupancyDerivative}(n)$ 
22:   if low <  $c$  < high then
23:      $n_{\text{next}} \leftarrow c$ 
24:   else
25:      $n_{\text{next}} \leftarrow (\text{low} + \text{high})/2$ 
26:   end if
27: until  $|n_{\text{next}} - n|/n \leq 10^{-12}$ 
28: return  $n_{\text{next}}$ 

```

The computational cost is $O(2^B)$ per estimate: the sums in (1) and (2) have $O(2^B)$ terms, and the Newton loop converges in a bounded number of iterations (typically 5 to 10). The cost depends on neither n nor $m = 2^P$, and the stored entries are never scanned.

Table 1: Aggregate error over the sparse regime ($P = 10$, $B = 6$, 256 seeds). MAE is mean absolute error in elements, MRE is mean signed relative error, MSE is mean squared error, and MARE is mean absolute relative error (cardinality-weighted over the sparse regime). The baseline register estimate uses linear counting in this range.

Estimator	MAE	MRE	MSE	MARE
Uncorrected (D)	208.1	-13.98%	75551	13.98%
Registers (linear counting)	25.8	+0.17%	1432	2.20%
Registers (MLE)	22.4	+0.12%	1052	1.98%
Corrected (occupancy inverse)	11.3	-0.02%	292	0.85%
SetSketch ($b=2$)	35.4	-0.01%	2332	3.83%
SetSketch ($b=1.2$)	35.4	-0.02%	2324	3.83%

4 Empirical Results

We evaluate the method at precision $P = 10$ ($m = 1024$ registers) and register width $B = 6$ ($R = 63$). The sparse regime spans cardinalities from approximately $n = 52$ (end of the value list) to $n \approx 2153$ (conversion to registers). This range falls within the linear-counting regime of the register estimator (threshold $n = 2320$ for this configuration), and the HLL++ empirical bias-correction tables do not engage until $n > 2320$. For each cardinality, we report the mean absolute relative error over 256 random seeds.

The corrected estimate is $2.3\times$ better than linear counting on MAE and $4.9\times$ on MSE. The register MLE estimator of Ertl [4] improves over linear counting (MAE 22.4 vs 25.8, MARE 1.98% vs 2.20%) but remains $2.3\times$ worse than the corrected hash list on MARE (1.98% vs 0.86%). SetSketch with the same bucket count ($m = 1024$) achieves MAE of 35.4 across all tested b values, roughly $1.4\times$ worse than linear counting and $4.5\times$ worse than the corrected estimate. All three correction methods (MLE, occupancy inverse, and SetSketch) are effectively unbiased (MRE near zero), while the uncorrected distinct count is biased low by 14% due to collisions.

The critical distinction between the hash list and SetSketch appears at low cardinality. The three-stage design provides exact cardinality through the value-list stage ($n \leq 52$) and near-exact estimates in the early hash-list regime (MARE below 0.01%). SetSketch has no exact-counting mode: it is purely stochastic from $n = 1$, producing MARE of 69% at $n = 1$, and only reaching the 3% range beyond $n = 500$. This structural gap means SetSketch cannot replace a multi-stage adaptive representation when low-cardinality accuracy matters.

5 Relation to HyperLogLog++

HLL++ [2] (deployed in Google’s infrastructure) introduces three improvements over the original HLL: a 64-bit hash function, a sparse representation for low cardinality, and an empirically fitted bias-correction table. The HLL++ sparse representation and our compressed sparse representation share the same fundamental idea: defer the register array by storing explicit hash data in a compressed sorted list. HLL++ stores index-rank pairs (one per bucket, residuals discarded); the hash list stores composites (index plus rank plus residuals, multiple per bucket).

What HLL++ stores. The HLL++ sparse representation keeps one entry per occupied bucket: a pair of bucket index and rank ($\text{idx}, \rho(w)$) at elevated precision ($p' = 25$). If two elements hash

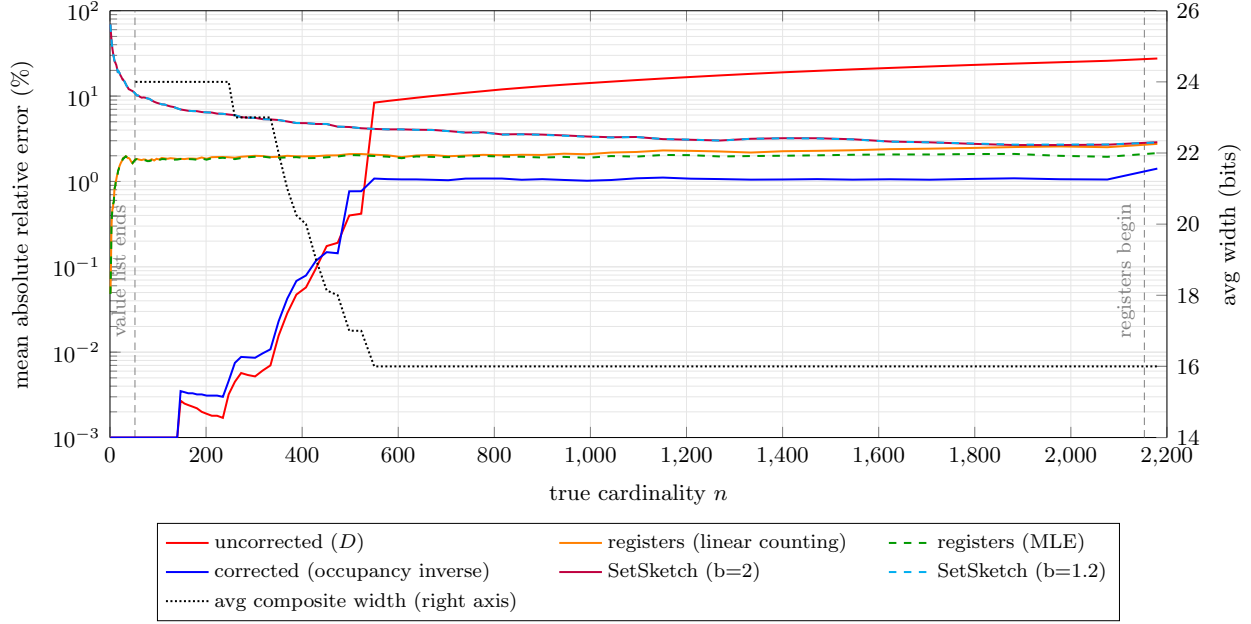


Figure 1: Mean absolute relative error over 256 seeds ($P = 10$, $B = 6$). The uncorrected count D is biased low. The register baseline uses linear counting. The occupancy inverse stays below both. SetSketch (same $m = 1024$) has higher error and lacks an exact-counting mode. The dotted curve (right axis) is the average composite width, falling from 24 to 16 bits. Dashed verticals mark representation transitions.

to the same bucket, only the one with the higher rank survives. The residual hash bits beyond the index and rank are discarded. The sorted list is compressed with variable-length integer encoding and difference coding, using roughly 8 bits per entry at $P = 14$. When the compressed list exceeds $6m$ bits in size, HLL++ converts to the dense register array. At query time, if the representation is still sparse, linear counting on the number of distinct indices gives the estimate.

What the hash list stores. The hash list keeps distinct *composites*: each entry concatenates the P -bit bucket index, the register rank, and *residual hash bits* from the original hash. Two elements with different ranks both survive as separate composites. The sorted distinct composites are packed with Rice-coded gaps between consecutive entries. The composite width w starts large (e.g. 24 bits at $P = 10$) and shrinks as the list fills, making the gaps smaller and the Rice codes shorter.

Why retaining residual bits matters. By keeping residual hash bits, the hash list carries more cardinality per byte than HLL++’s sparse representation. The Rice-coded gap compression makes this affordable: at $P = 10$ and $w = 24$, each composite costs 3-4 bytes. The wider composite space (2^{24} versus 2^{14} buckets) is offset by Rice coding over small gaps. The extra information is what pushes the representation much further: the hash list carries cardinality deep into the collision regime (at $P = 10$), where HLL++ would have already converted to registers ($n \approx 534$ at $P = 10$, Heule et al. [2]).

Why pushing further requires bias correction. Because the hash list reaches cardinalities where n approaches the composite space 2^w , collisions between distinct elements producing the

same composite become frequent. The raw distinct count D underestimates n by over 14% near conversion. The occupancy-inverse correction is what makes this regime usable: it models the non-uniform collision structure of the composite encoding and inverts the expected distinct count to recover n . HLL++ does not face this problem because its sparse representation converts to registers before collisions become significant, relying instead on linear counting (which is accurate in the narrow band where HLL++ operates) and its empirical bias tables for the intermediate regime.

Complementarity. The hash list and HLL++ are complementary. The hash list covers the linear-counting regime of the register estimator ($n \lesssim 2153$ at $P = 10$), where HLL++ relies on linear counting and empirical bias tables. The hash list provides a table-free alternative with lower error across the entire regime.

6 Discussion: SetSketch and HyperLogLogLog

SetSketch [3] generalizes both HLL and MinHash by storing per-bucket maximums of exponentially distributed hash values instead of uniform ranks. A continuous parameter b interpolates between the HLL limit ($b = 2$, HLL-equivalent register statistics) and the MinHash limit ($b \rightarrow 1$, equivalent to MinHash via a monotone transformation). Its cardinality estimator uses a closed-form harmonic-mean variant with analytically derived corrections for small and large cardinalities, requiring no empirical calibration. We evaluate SetSketch across three b values: $b = 2$ (HLL limit), $b = 1.2$, and $b = 1.001$ (near the MinHash limit).

Our empirical comparison (Figure 1, Table 1) evaluates SetSketch with the same bucket count ($m = 1024$) against the hash list. In the sparse regime SetSketch achieves MARE of 3.83% across all tested b values, higher than both linear counting (2.20%), the register MLE (1.98%), and the occupancy inverse (0.86%). The different b values produce nearly identical results, confirming the estimator is insensitive to this parameter in the sparse regime. The dominant limitation is low-cardinality accuracy: SetSketch lacks an exact-counting mode and is purely stochastic from $n = 1$, yielding MARE of 69% at $n = 1$ and 12% at $n = 50$. The three-stage design avoids this entirely through the value-list stage (exact through $n = 52$) and the near-exact early hash-list regime.

The occupancy model of the hash list is not directly applicable to SetSketch because its register values follow an exponential distribution rather than the non-uniform composite encoding structure. An adapted model would need a separate derivation for the order-statistic distribution of per-bucket maxima under exponential hashing.

6.1 HyperLogLogLog

HyperLogLogLog [5] compresses the HyperLogLog register array by storing a base value B and per-register offsets $M[j] - B$ in a dense array, with overflowing registers kept sparsely as $(j, M[j])$ pairs. This reduces the sketch from $O(m \log \log n)$ bits to $m \log_2 \log_2 \log_2 n + O(m + \log \log n)$ bits while preserving all estimation properties and mergeability in the compressed domain.

The hash list and HyperLogLogLog address different regimes: HyperLogLogLog compresses the register representation at high cardinality, while the hash list replaces registers entirely at low cardinality. The two techniques are complementary: the hash list defers the register abstraction, and HyperLogLogLog compresses it when it is eventually needed.

7 Conclusion

We presented a three-stage adaptive representation for cardinality estimation that achieves sub-register error in the low-cardinality regime. The hash list with Rice-coded gaps and bias correction reduces mean absolute relative error by $2.6\times$ over linear counting, $2.3\times$ over register MLE, and $4.5\times$ over SetSketch. The MLE approach of Ertl [4] narrows the gap between linear counting and the corrected estimate (MARE 1.98% vs 2.20% vs 0.86%) but cannot overcome the fundamental limitation: at low cardinality the register statistics are dominated by zeros, and no post-hoc correction on the register array alone can recover the lost information. The three-stage design (value list, hash list, then dense representation) avoids this structural gap entirely by keeping data explicit until compression is unavoidable. The counter converts to HLL registers when the hash list saturates, so it drops directly into existing HLL++ pipelines. The $O(d)$ insert cost in the hash list is the primary limitation, where d is the number of stored distinct composites: each insertion requires a sorted splice into the gap-encoded bitstream, which involves decompressing, inserting, and recompressing a portion of the list. Because the Rice-coded packing efficiency depends on the gap distribution of the input hashes, the estimator variance is data-dependent rather than a function of cardinality alone. Future work includes deriving an occupancy model for SetSketch’s exponential register distribution and formalizing the occupancy theorem in a proof assistant (e.g. Lean) to guarantee correctness of the cell probability assignments. The implementation is available as an open source crate.²

References

- [1] Philippe Flajolet, Eric Fusy, Olivier Gandouet, and Frederic Meunier. HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm. In *Proceedings of the 2007 conference on analysis of algorithms*, pages 127–146, 2007.
- [2] Stefan Heule, Marc Nunkesser, and Alexander Hall. HyperLogLog in practice: algorithmic engineering of a state of the art cardinality estimation algorithm. In *EDBT/ICDT*, pages 3–14, 2013.
- [3] Otmar Ertl. SetSketch: filling the gap between MinHash and HyperLogLog. *Proceedings of the VLDB Endowment*, 14(11):2244–2257, 2021.
- [4] Otmar Ertl. Probabilistic analysis of the HyperLogLog algorithm. *Algorithmica*, 79(2):519–535, 2017.
- [5] Matti Karppa and Rasmus Pagh. HyperLogLogLog: cardinality estimation with one log more. *arXiv preprint arXiv:2205.11327*, 2022.
- [6] Robert G. Gallager and Philip C. van Voorhis. Optimal noninteger expansions of digital data. *Bell System Technical Journal*, 54(8):1415–1423, 1975.
- [7] Solomon W. Rice. Class of two-dimensional transform coding techniques. *Bell System Technical Journal*, 38(3):411–432, 1959.
- [8] Paolo Boldi, Marco Rosa, Matteo Santini, and Sebastiano Vigna. Layered label propagation: a unified approach to hierarchy extraction and ordering in graphs. *arXiv preprint arXiv:1011.5425*, 2010.

²<https://github.com/LucaCappelletti94/hyperloglog-rs>

- [9] Paolo Boldi and Sebastiano Vigna. In-core computation of geometric centralities with HyperBall: a hundred billion nodes and beyond. *arXiv preprint arXiv:1308.2144*, 2013.